ECE 315 Lab 1: Interfacing to Input/Output Devices

Authors: Yaaqoob Choulli, Rowan Rasmusson

Student ID: 1752634, 1757167

CCID: yaaqoob, rrasmuss

Course: ECE 315

Lab Section: H21

Date Performed: February 4, 2026

Date Due: February 24, 2026

Abstract:

The goal of the first lab in computer interfacing is to familiarize and implement a small, functioning, architecturally sound FreeRTOS system. The objectives to complete this were to have a task capable of adjusting the brightness of an LED using pulse width modulation, and have numbers from a keypad show up on a seven-segment display. These objectives were to be first implemented using one large task for each piece of functionality. But later, we implemented them with queues to be more robust, efficient with their time slice, and idiomatic of a true FreeRTOS system. The hardware this was implemented on was a Zybo Z7-10 development board on a Zynq 7000 processor.

Design:

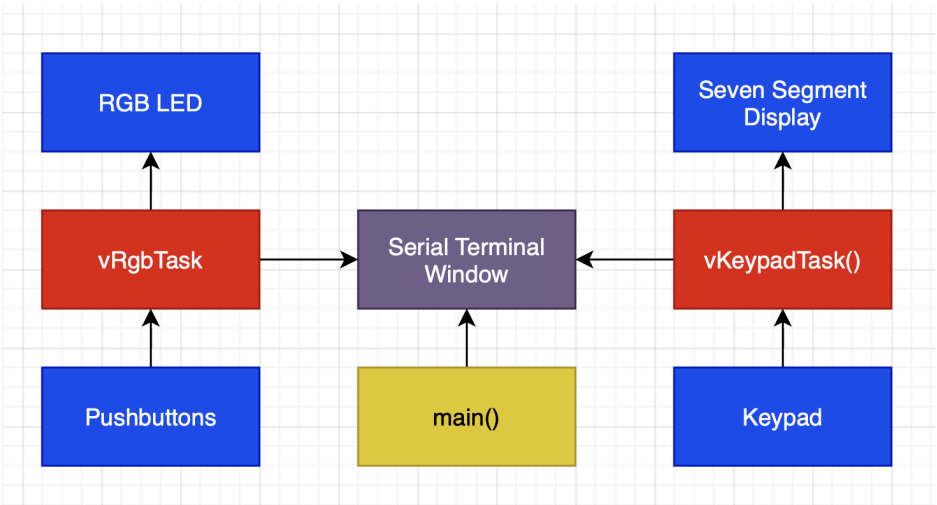

Figure 1: Initial Architectural Diagram. This figure shows the initial/preliminary architecture of the system, where there are 4 peripherals in blue, and two tasks in red. This picture comes from the prelab.

Part 1:

The objective for part 1 was to set up the hardware in Vivado to create the hardware configuration .xsa file for the Zybo board, modify the BSP to fit our needs, and get values to appear on the SSD (seven-segment display) when pressed on the keypad. The BSP was modified to increase the *freertos_tick_rate* to 1000 ticks per second. This was done to decrease the amount of wall-clock time between task delays. In other words, updating the RTOS tick resolution so each tick is 1 millisecond. This increases the granularity and provides us with faster round robin scheduling with equal priority tasks..

Next, a task for interfacing with the keypad and the SSD was created. It communicated with the peripherals using the GPIO PMOD ports on the Zybo Z7. The task was in charge of taking data given from the input of the keypad, decoding it to tell which value to display, and then sending it over the GPIOs to be displayed. Part of its duty cycle was listening to the input from the keypad, and upon a keypress, updating its internal values of *previous_key* and *current_key*. These two variables allowed for 2 values to be displayed on the SSD at once. As part of its duty cycle, the keypad task would also send the *previous_key* and *current_key* to the SSDInst variable for it to decode and display. This is shown below in *Figure 2*.

```c
        while (1){
            // Capture state of the keypad
            keystate = KYPD_getKeyStates(&KYPDInst);

            // Determine which single key is pressed, if any
            // if a key is pressed, store the value of the new key in new_key
            status = KYPD_getKeyPressed(&KYPDInst, keystate, &new_key);
            // Print key detect if a new key is pressed or if status has changed
            if (status == KYPD_SINGLE_KEY && previous_status == KYPD_NO_KEY){
                xil_printf("Key Pressed: %c\r\n", (char) new_key);
    /************************** Enter your code here **************************/
                // TODO: update value of previous_key and current_key
                previous_key = current_key;
                current_key = new_key;

    /*****************************************************************************/
            } else if (status == KYPD_MULTI_KEY && status != previous_status){
                xil_printf("Error: Multiple keys pressed\r\n");
            }

    /************************** Enter your code here **************************/
            // TODO: display the value of `status` each time it changes
            xil_printf("Status %u", status);

    /*****************************************************************************/
```

Figure 2: This is code that shows the task getting input from the keypad and assigning the interval variables accordingly.

```c
    /***************************** Enter your code here *****************************/
            /* TODO: Decode the current and previous keys using the `SSD_decode` function.
            * Write each decoded value to the seven-segment display, one at a time,
            * using the `XGpio_DiscreteWrite` function.
            * Add a delay between updates for persistence of vision using `vTaskDelay`.
            */
            ssd_value = SSD_decode(current_key, 0);
            XGpio_DiscreteWrite(&SSDInst, 0u, ssd_value);

            vTaskDelay(xDelay);

            ssd_value = SSD_decode(previous_key, 1);
            XGpio_DiscreteWrite(&SSDInst, 1u, ssd_value);

            vTaskDelay(xDelay);
    /*****************************************************************************/
```

Figure 3: This second picture of the section shows the decoded keys from the keypad being sent over the GPIOs to be displayed on the SSD. The first argument is the declared XGpio instance for the SSD, the middle argument is the channel of the XGpio to write to, two different ones for the different values, and the last is the bit mask to write to the SSD.

Part 2:

Part 2 was a continuation of the prior, where we leveraged PWM or pulse width modulation signalling to control the brightness of our LED. Initially, in part 1, depending on the button pressed we adjusted the frequency of the flashing LED to determine the ideal delay period for the human eye to perceive a solid on light. For this part of the lab, we maintained the same period however, we divided it into two components, xOnDelay and xOffDelay. The sum of these two values will always be equal to the period determined in part 1, '*14'*. To increase the brightness of the LED we increase the value of xOnDelay, while decreasing the value of xOffDelay to maintain the fixed delay period constraint. We can see this below on *Figure 4,* on lines 217, 219, 220. On the other hand, to decrease the brightness, we do the opposite: we increase the xOffDelay, and simultaneously decrease the xOnDelay by the same amount. This is found on *Figure 4,* on lines 210, 212, 213.

```
210             if (val == 8) {
211                 if (!(xOnDelay < 1)) {
212                     xOnDelay -= 1;
213                     xOffDelay += 1;
214                 }
215                 xil_printf("\nOffDelay=%u, OnDelay=%u\n", xOffDelay, xOnDelay);
216                 vTaskDelay(50UL);
217             } else if (val == 1) {
218                 if (!(xOffDelay < 1)) {
219                     xOnDelay += 1;
220                     xOffDelay -= 1;
221                 }
222                 xil_printf("\nOffDelay=%u, OnDelay=%u\n", xOffDelay, xOnDelay);
223                 vTaskDelay(50UL);
224             }
```

Figure 4: This figure shows the code for the LED Pulse Modulation discussed above.

Part 3:

The objective of part 3 was to maintain all the logic from the prior parts however, refactor the code. In its initial state, our design used two tasks, vKeypadTask and vRgbTask. These tasks handled all the logic related to the keypad, seven-segment displays, pushbuttons, and an LED. With that being said, this is poor architectural design since a task should do one thing through the idea of separation of concerns. Thus, we started by adding two new tasks, vButtonsTask, vDisplayTask, and two new queues, xRgbLedQueue, and xDisplayQueue for inter-task communication.

The original vRgbTask was changed to receive the xOnDelay and xOffDelay values from the xRgbLedQueue and then turn on/off the LED. It no longer contains logic to handle the button pressing; this has been moved to the new vButtonsTask, where it determines the xOnDelay and xOffDelay and puts the respective data on the xRgbLedQueue.

The original vKeypadTask would handle the logic of receiving input from the keypad, then displaying it in the seven segment display (SSD). The displaying logic has been extracted and placed into the new vDisplayTask. This consumes the current_key and previous_key from the new xDisplayQueue, then

writes them into the SSD instance. Now the vKeypadTask only receives input from the keypad instance and places captured data on the xDisplayQueue.

With these changes for part 3, each task has one isolated job and leverages a queue to communicate between the jobs. The code for all 4 tasks can be found as part of the other submission files.

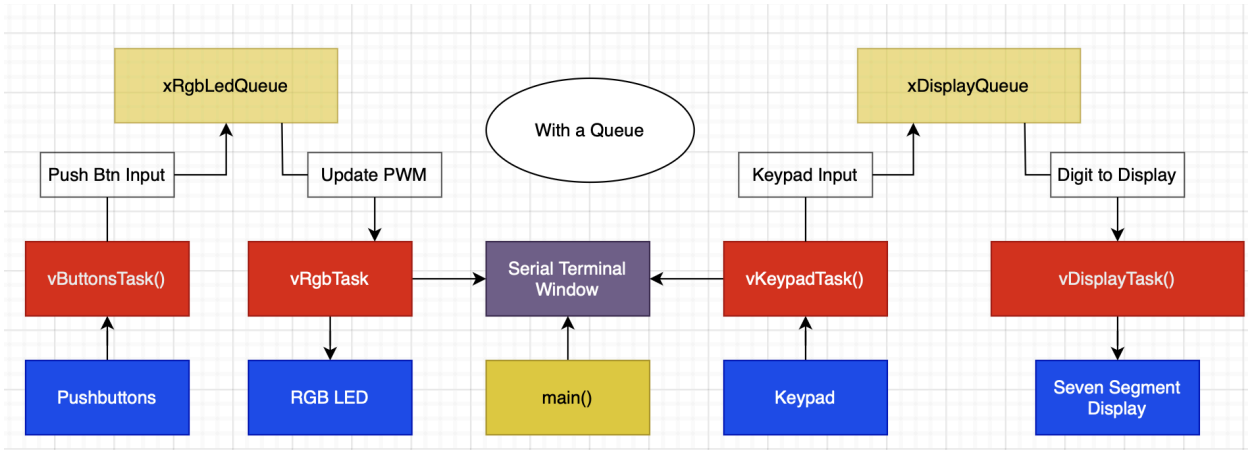

Figure 5: This figure shows the complete Architectural Diagram. As per the goal of part 3, we now have 4 *Tasks* in red (two new ones), and two new *Queues* shown in light yellow at the top.

<u>Discussion:</u>

Part 1:

Part 1 was quite straightforward; however, we ran into the issue where the second digit on the seven-segment display wasn't showing up. To address this, our solution was to add an extra delay at the end of ssd task duty cycle, specifically after the previous_key was written, seen on line 151 in *Figure 6.*

```
143            ssd_value = SSD_decode(current_key, 0);
144            XGpio_DiscreteWrite(&SSDInst, 0u, ssd_value);
145
146            vTaskDelay(xDelay);
147
148            ssd_value = SSD_decode(previous_key, 1);
149            XGpio_DiscreteWrite(&SSDInst, 1u, ssd_value);
150
151            vTaskDelay(xDelay);
```

Figure 6: This figure shows the code belonging to the vKeypadTask

Part 2:

The main problem we had with part 2 was that the LED was not turning off completely when reaching the minimum PWM cycle. Adjusting the T_{on} part of the cycle to zero should turn it completely off, but ours stayed on very faintly. To solve this, we used the code below:

```
    if (xOnDelay != 0) {
        XGpio_DiscreteWrite(&rgbLedInst, RGB_CHANNEL, color);
        vTaskDelay(xOnDelay);
    }
```

Figure 7: xOnDelay is the variable holding the portion of the LEDs period that it should be on. This code ensures that if xOnDelay ever reaches zero, we shouldn't turn it on. Otherwise, it always turns it on, even for a short time representing a dim LED.

Part 3:

In part 3, we had a couple speedbumps while setting up the queues for communication between tasks. The first of these issues was that we initiated one of the buffers to NULL. Before we fixed it, in the image below image the recvd_packet was equal to NULL. This resulted in a pointer constant being assigned to a non-pointer type. Thus, xQueueReceive() could not read the information into the displayPacket struct as intended.

```
displayPacket recvd_packet;
const TickType_t xDelay = pdMS_TO_TICKS(DELAY);
u32 ssd_value=0;
u8 current_key = 'x', previous_key = 'x';

while(1) {
    if (xQueueReceive(xDisplayQueue, &recvd_packet, ReadTimeout) == pdTRUE) {
        current_key = recvd_packet.current_key;
        previous_key = recvd_packet.previous_key;
    }
```

Figure 8: Showing where we wrongly initialized recvd_packet to NULL.

Also in figure Y, we had the ReadTimeout set to 100ms, which was problematic because we had our DELAY constant used to get xDelay set to 10ms. This caused the task to become more event-driven than periodic since it was waiting for an order of magnitude larger time than the value it used to delay itself for vTaskDelay() later in the code. Since the values were not being updated on the display consistently upon keypad pressing, we hypothesize that the large timeout ended up hogging a lot of the scheduler timing, resulting in the keypad listening task not consistently picking up the pressed keys.

Testing Methodology:

For testing, we manually tried different combinations of inputs on the keypad and buttons controlling the LED. All these tests passed. Some test combinations for the SSD and LED include:

Test	Expected Output	Rationale
SSD		
Inputting the same number twice	See both numbers displayed on the SSD	Ensure the correct base behaviour of the keypad + SSD system having two of the same input.
Input 2 different numbers	See both numbers displayed on the SSD	Ensure the correct base behaviour of the keypad + SSD system having two different inputs.
1 number by itself at the beginning, expecting the other side to remain blank	See the left side blank and the right side displaying the number from the keypad	Make sure that the SSD is able to display a lack of input / blank input at the beginning of operation
Holding one keypad number down. As intended, this only resulted in one instance of that value being displayed	Not having repeated values rapidly being printed to the SSD	Make sure the user cannot rapidly overwrite the SSD output with other numbers. This was decided as expected behaviour by us.
LED		
Turning the LED the brightest	The LED no longer keeps getting brighter and reaches its max brightness when pushing the '8' button.	Make sure that there is a threshold for the brightness implemented, and no unexpected behaviour happens at this edge case
Turning the LED off	LED turns off upon reaching the minimum OnDelay period.	Make sure that there is a threshold for the LED to turn off, and no negative values for the period are reached
Pushing the middle 'push buttons' ie. a number that is not 8 or 1	No change	Ensure that no unexpected input / random fiddling can cause system failure
Alternating pushing 1 and 8 to rapidly test increasing and decreasing the brightness	See the LED change brightness from bright to dim rapidly	Ensure the system can withstand fast operator inputs for the LED task to meet real-time constraints.
Pushing the push buttons while simultaneously clicking the keypad.	Watch values show up on the SSD while seeing the	Ensure the system can withstand fast operator inputs into both tasks to meet

	LED adjust its brightness	real-time constraints.

Conclusion:

The objective of this first lab was to both gain familiarity with FreeRTOS while also learning some of the fundamentals regarding peripherals and task communication/synchronization via queues. This took place in three parts, where the final goal was to make a seven-segment display show numbers taken from an input keypad, and be able to adjust the brightness of an LED by clicking different push buttons. All in all, this lab was a big success. We were able to achieve functionality in both aspects, in addition to a newfound understanding of FreeRTOS, tasks, queues, and peripheral debugging. Although we came across some minor difficulties in the lab, we successfully overcame them and tested our system thoroughly through the test matrix above. Ultimately, this first ECE 315 lab was done in full and was a complete success.

References:

[1] Provided lab documentation and source code
[2] Inclass notes
[3] FreeRTOS Documentation; https://freertos.org/Documentation/00-Overview